Software Engineering Lab

Lab 16

Dependency Injection

Memisahkan Konfigurasi dari Penggunaan untuk Testability & Evolvability



Lukman / lukman-ss

Infrastruktur Terkunci di Logika Bisnis

Ketika modul bisnis membuat dependensinya sendiri, isolasi dan pengujian runtuh.

MASALAH PRODUKSI

Tightly Coupled Code

Processor menginstansiasi RealGateway langsung di method; logika bisnis terkunci ke implementasi vendor tunggal.

MASALAH PRODUKSI

Pengujian Lambat & Flaky

Unit test terpaksa membuka koneksi jaringan nyata atau memerlukan mock REST API server yang lambat dan rapuh.

MASALAH PRODUKSI

Ripple Effect Perubahan

Mengganti vendor payment gateway memaksa refactoring dan rekompilasi menyeluruh di seluruh lapisan consumer.

Kopling keras antara logika domain dan implementasi infrastruktur merusak evolvability sistem. Pengujian unit menjadi mustahil dilakukan secara murni tanpa network overhead.

Hard-coded dependencies fuse domain logic to external plumbing, destroying testability.

Review Manual Melewatkan Hidden Dependencies

Service Locator tampak bersih di permukaan namun menyembunyikan kontrak asli.

KELEMAHAN PENDEKATAN NAIF

Kontrak Opaque

Signature NewBadProcessor(c Container) tidak menyatakan butuh PaymentGateway; dependensi baru terlihat di dalam baris method.

KELEMAHAN PENDEKATAN NAIF

Mocking Menjadi Rumit

Test tidak bisa menyuntikkan MockGateway langsung, melainkan harus membuat MockContainer yang membungkus MockGateway.

KELEMAHAN PENDEKATAN NAIF

Ledakan Runtime

Jika container lupa meregistrasikan gateway, compiler tetap lolos namun aplikasi meledak panic saat dipanggil di production.

Reviewer manusia melihat parameter Container dan menganggap modularitas tercapai. Faktanya, Service Locator hanya memindahkan kopling dari compile-time ke runtime.

Service Locator hides dependencies inside methods; constructor injection makes them explicit at creation.

Separation of Configuration from Use

Fowler: fase perakitan objek terpisah total dari fase eksekusi logika bisnis.

MAIN.GO / ASSEMBLER

Fase Konfigurasi (Composition Root)

Bertanggung jawab membuat objek konkret (RealGateway), merakit relasi, dan menginjeksikan ke consumer.

PROCESSOR (BUSINESS SERVICE)

Fase Penggunaan (Domain Execution)

Hanya menerima PaymentGateway via interface; mengeksekusi ProcessPayment tanpa tahu kelas konkret di baliknya.

> Prinsip paling fundamental dari DI bukanlah framework atau container, melainkan pemisahan waktu pembuatan objek dari waktu penggunaan objek.

Assemble the graph once at startup; execute pure business logic during runtime.

Pipeline Alur Eksekusi Dependency Injection

Dari entry point aplikasi hingga eksekusi infrastruktur terabstraksi.

STEP 01

cmd/demo/main.go

Composition Root inisialisasi RealGateway

→

STEP 02

di.NewProcessor(gateway)

Konstruktor menerima kontrak PaymentGateway

→

STEP 03

processor.ProcessPayment(100)

Validasi input amount <= 0 & buat Money

→

STEP 04

gateway.Charge(Money)

Delegasi eksekusi ke implementasi konkret

Pipeline memperlihatkan batasan tanggung jawab yang jelas: composition root menangani konstruksi, processor menangani validasi, dan gateway menangani charge.

Separate configuration from use: simple linear stages create decoupled architecture.

Constructor Injection: Valid State at Birth

Field unexported, dependensi eksplisit, dan objek dijamin lengkap.

```
type Processor struct {
    gateway PaymentGateway
}

func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}
```

Dengan Constructor Injection, Processor tidak mungkin dibuat dalam keadaan setengah matang. Signature constructor menyatakan secara gamblang apa yang dibutuhkan.

Explicit constructor parameters enforce immutable, valid instances at creation time.

Failure Path: Pencegahan Eksekusi I/O Sia-sia

Validasi domain memotong eksekusi sebelum memanggil payment gateway.

STEP 01

Input Transaksi

amount = -10 (invalid input)

STEP 02

Processor Pre-check

if amount <= 0 { return error }

STEP 03

Short-Circuit Exit

Return errors.New("invalid amount")

STEP 04

Infrastruktur Aman

gateway.Charge() tidak pernah dipanggil (Amount = 0)

> DI tidak menghilangkan kebutuhan validasi. Pre-check di Processor memastikan gateway tidak pernah dipanggil ketika parameter bisnis tidak valid.

Validate at domain boundary; never leak invalid data into infrastructure calls.

internal/di/processor.go:15-21 • tests/processor_test.go:47-59

Tabel Evaluasi Mekanisme Dependensi

Perbandingan struktural antara Constructor Injection, Service Locator, dan Hard-Coded.

Kriteria	Hard-Coded New	Service Locator	Constructor Injection
Transparansi Kontrak	Tertutup di method	Tertutup di container	Terbuka di constructor
Validasi Compiler	Kuat (tipe konkret)	Lemah (runtime get)	Sangat Kuat (interface)
Kompleksitas Test	Sangat tinggi (mock I/O)	Sedang (mock container)	Minimal (mock interface)
Standar Industri	Anti-pattern monolit	PSR-11: SHOULD NOT	Rekomendasi Fowler / Default

Constructor injection memberikan kombinasi terbaik: dependensi terlihat transparan, compiler memvalidasi pemenuhan interface, dan test double dapat diinjeksi tanpa overhead.

Constructor injection provides compiler-enforced transparency; service locator delays failures to runtime.

Tiga Keputusan Desain Dependensi

Urutan keputusan arsitektur untuk menjaga kemurnian boundary sistem.

01. KONTRAK INTERFACE

Program to Interface

Definisikan PaymentGateway Charge(Money) error untuk mengabstraksi seluruh varian vendor pembayaran.

02. WAJIBKAN CTOR INJECTION

Explicit Dependencies

Tolak Service Locator; oper PaymentGateway langsung ke konstruktor NewProcessor.

03. VALUE OBJECT DIRECT

Exempt Pure Data

Instansiasi Money{Amount, Currency} langsung tanpa DI container karena tidak memiliki efek I/O.

Keputusan ini memastikan bahwa abstraksi hanya diterapkan pada batas sistem eksternal, sementara struktur data murni tetap ringan tanpa beban overhead DI.

Interface external boundaries, inject at constructors, instantiate value objects directly.

Output Demo: Eksekusi Dua Pendekatan

Hasil run membuktikan kesamaan fungsional dengan perbedaan struktural total.

```
$ go run ./cmd/demo
--- Running Constructor Injection ---
RealGateway charging 100 USD
--- Running Service Locator ---
RealGateway charging 200 USD
```

Demo membuktikan bahwa kedua pola berhasil mendistribusikan pembayaran ke RealGateway, namun Constructor Injection mencapai hasil ini tanpa keterikatan pada API container.

Functional equivalence in execution; architectural superiority in isolation.

Batasan Bukti: Cakupan Verifikasi Lab

Memisahkan fakta terverifikasi laboratorium dari asumsi ekosistem produksi.

TERBUKTI DI LAB

YANG TERBUKTI DI LAB

- MockGateway mengisolasi Processor 100% tanpa jaringan.
- Validasi pre-check mencegah pemanggilan gateway.
- Error gateway dipropagasikan utuh ke caller.
- Race detector bersih tanpa data race.

TIDAK DIBUKTIKAN

YANG TIDAK DIBUKTIKAN

- Overhead performa container vs direct instantiation.
- Korelasi penurunan angka bug/defect di tim engineering.
- Lifecycle manajemen (Singleton, Scoped, Transient).
- Efektivitas DI framework otomatis (Wire, Dig) skala besar.

Tests verify isolation and contract adherence, not ecosystem-scale lifecycle management.

Trade-off: Kontrol Bersih vs Biaya Boilerplate

Keuntungan modularitas harus dibayar dengan disiplin perakitan dependensi.

KEUNTUNGAN DIPEROLEH

KEUNTUNGAN DIPEROLEH

- Isolasi total: unit test berjalan cepat dalam orde mikrodetik.
- Vendor agnostic: ganti gateway tanpa sentuh logika bisnis.
- Immutabilitas: dependensi tidak dapat diganti sembarangan di runtime.

KONSEKUENSI DIHADAPI

KONSEKUENSI DIHADAPI

- Manual wiring menambah baris inisialisasi di composition root.
- Tracing alur kode memerlukan navigasi antarmuka interface.
- Risiko constructor bloat jika kelas melanggar Single Responsibility.

Every decoupled boundary trades runtime simplicity for construction ceremony.

Pipeline Validasi & Thread-Safety

Processor berstatus stateless dan aman dieksekusi lintas goroutine paralel.

STEP 01

Goroutine Request

Panggilan concurrent ke ProcessPayment(amount)

STEP 02

Pure Pre-Validation

amount <= 0 dicek tanpa mutasi shared state

STEP 03

Value Object Build

Money{Amount, Currency} dialokasikan per request

STEP 04

Thread-Safe Charge

Gateway dieksekusi secara aman (Race Clean)

Karena Processor tidak menyimpan state yang dapat dimutasi (hanya menyimpan reference interface gateway yang stateless), pemrosesan pembayaran bebas dari race condition.

Immutable dependency references make services inherently safe for concurrent execution.

Sebelum vs Sesudah Penerapan DI

Transformasi kode rapuh menjadi komponen yang teruji dan adaptif.

BEFORE: NAIVE / UNPROTECTED

SEBELUM: TIGHTLY COUPLED

- Processor memanggil new RealGateway() secara internal.
- Unit test terpaksa menghubungi server gateway nyata.
- Pergantian vendor menuntut perubahan di logika bisnis.
- Sulit menguji skenario kegagalan jaringan secara deterministik.

AFTER: OPTIMIZED / RESILIENT

SESUDAH: CONSTRUCTOR INJECTION

- Processor menerima PaymentGateway via NewProcessor.
- Unit test menyuntikkan MockGateway tanpa sentuh jaringan.
- Pergantian vendor hanya mengubah satu baris di main.go.
- Skenario kegagalan gateway dapat diuji deterministik 100%.

Decoupled interfaces turn brittle integration tests into ultra-fast deterministic unit tests.

Test Suite 01: Verifikasi Happy Path Terisolasi

Pengujian mengeksekusi transfer 50 USD tanpa membuka koneksi jaringan.

VERIFIKASI SUKSES

Substitusi Bersih

mock := &MockGateway{} disuntikkan ke NewProcessor(mock); tidak ada panggilan TCP atau setup database.

VERIFIKASI SUKSES

Validasi Status Mutasi

mock.ChargedMoney mencatat persis Amount: 50 dan Currency: 'USD' setelah ProcessPayment(50) berhasil.

VERIFIKASI SUKSES

Zero-Latency Run

Eksekusi test selesai dalam 0.00s deterministik, bebas dari network flakiness atau rate-limit vendor.

Test happy path membuktikan bahwa Processor mendelegasikan data secara akurat ke gateway tanpa perlu mengetahui implementasi di belakang interface.

Mock substitution proves delegation accuracy without touching concrete infrastructure.

tests/processor_test.go:23–35

Test Suite 02: Propagasi Kegagalan Gateway

Memverifikasi error dari infrastruktur diteruskan utuh tanpa ditelan.

VERIFIKASI PROPAGASI ERROR

Simulasi Down Time

MockGateway dengan ShouldFail: true mensimulasikan kondisi 'gateway unavailable' secara instan.

VERIFIKASI PROPAGASI ERROR

Error Forwarding

Processor mengembalikan error gateway langsung ke caller tanpa menelan (swallow) pesan kesalahan.

VERIFIKASI PROPAGASI ERROR

Audit Integritas

Caller menerima sinyal kegagalan deterministik untuk memicu mekanisme retry atau fallback otomatis.

> Menguji kegagalan gateway pada arsitektur tightly coupled sangat sulit. Dengan DI, kegagalan vendor dapat disimulasikan hanya dengan mengaktifkan flag pada test double.

Deterministic error simulation verifies resilient error propagation paths.

Test Suite 03: Short-Circuit Invalid Amount

Memastikan gateway tidak pernah dipanggil jika parameter nominal salah.

VERIFIKASI SHORT-CIRCUIT

Input Negatif / Nol

ProcessPayment(-10) menghasilkan error 'invalid amount' seketika pada boundary validasi.

VERIFIKASI SHORT-CIRCUIT

Assertion Gateway Call

Assertion mock.ChargedMoney.Amount == 0 membuktikan Charge() tidak pernah disentuh sama sekali.

VERIFIKASI SHORT-CIRCUIT

Cost & Traffic Saving

Mencegah overhead biaya API call vendor dan transmisi data yang ditakdirkan gagal sejak awal.

Kombinasi assertion error dan pemeriksaan nilai 0 pada mock membuktikan bahwa validasi pre-check berhasil menghentikan alur sebelum mencapai lapisan infrastruktur.

Asserting zero interaction with downstream services proves boundary validation effectiveness.

tests/processor_test.go:47-59

Desain Rasional: Value Object Bebas DI

Money dan DTO murni dibuat langsung tanpa melewati container injector.

PRINSIP VALUE OBJECT

Data Murni Tanpa Efek

Struct Money{Amount, Currency} tidak berinteraksi dengan database, network, atau state eksternal.

PRINSIP VALUE OBJECT

Bahaya Over-Injection

Memasukkan struct data ke dalam container DI menambah layer indirection tanpa keuntungan testability.

PRINSIP VALUE OBJECT

Heuristik Desain

Gunakan DI untuk Service ber-efek samping (I/O); gunakan direct instantiation untuk data value murni.

Kesalahan umum pemula adalah menginjeksikan segalanya. Value object seperti Money, Date, atau Alamat adalah representasi data murni yang harus diinstansiasi langsung.

Inject services that cross boundaries; instantiate pure value objects directly.

internal/di/gateway.go:5–9 • internal/di/processor.go:19

Bahaya Service Locator: PSR-11 Warning

Menyuntikkan container ke domain menyamarkan dependensi dan merusak pengujian.

DAMPAK NEGATIF SERVICE LOCATOR

Pelanggaran PSR-11

Standar PHP-FIG PSR-11 menegaskan: 'Users SHOULD NOT pass a container into an object'.

DAMPAK NEGATIF SERVICE LOCATOR

Double-Mocking Overhead

Pengujian BadProcessor mewajibkan pembuatan MockContainer hanya untuk mengembalikan MockGateway.

DAMPAK NEGATIF SERVICE LOCATOR

Kopling ke API Container

Kelas bisnis menjadi terikat pada method GetPaymentGateway() milik container, bukan antarmuka generik.

> Service Locator adalah ilusi decoupling. Alih-alih terikat pada RealGateway, kelas justru terikat erat pada API Container yang lebih rumit untuk di-mock.

Passing a container to domain logic replaces one tight coupling with an even broader one.

Checklist Adopsi Dependency Injection

Panduan langkah praktis menerapkan DI yang bersih pada proyek produksi.

- ✓ Deklarasikan dependensi sebagai interface, bukan tipe struct konkret.
- ✓ Gunakan Constructor Injection sebagai standar default; tolak setter opsional.
- ✓ Pusatkan instansiasi objek konkret hanya pada Composition Root (main.go).
- ✓ Jangan pernah mengoper Container atau Service Locator ke dalam domain logic.
- ✓ Instansiasi struct data murni (Value Objects) langsung tanpa melalui DI.
- ✓ Pastikan setiap method bisnis memiliki unit test dengan test double terisolasi.

Follow a disciplined checklist to keep architectural boundaries clean and tests fast.

Tiga Anti-Pattern Umum Seputar DI

Kesalahan arsitektural yang sering ditemui saat mengadopsi pola injeksi.

ANTI-PATTERN UTAMA

Constructor Bloat

Konstruktor dengan belasan parameter adalah sinyal kuat pelanggaran Single Responsibility (SRP).

ANTI-PATTERN UTAMA

Service Locator Disguise

Membungkus dependensi ke dalam satu Map atau Container serbaguna demi menyederhanakan signature ctor.

ANTI-PATTERN UTAMA

Over-Abstracting Everything

Membuat interface untuk setiap kelas tanpa pernah memiliki lebih dari satu implementasi atau kebutuhan mock.

> DI bukan alasan untuk membiarkan sebuah kelas menangani terlalu banyak hal. Jika parameter konstruktor membengkak, solusinya adalah memecah kelas, bukan memakai Service Locator.

Constructor bloat is a design warning: split the class, don't hide the bloat in a container.

Reproduksi Mandiri Suite Pengujian

Verifikasi seluruh invarian dependency injection secara mandiri di mesin lokal.

```
cd labs/16-dependency-injection
go test -v ./...       # Verifikasi 6 skenario test suite
go test -race ./...    # Verifikasi thread-safety zero data race
go run ./cmd/demo      # Eksekusi demo komparasi Ctor vs Locator
```

Lab ini sepenuhnya berbasis Go standard library tanpa external dependencies. Seluruh verifikasi dapat direproduksi dalam hitungan detik di lingkungan apa pun.

Zero-dependency standard tests guarantee instant, reproducible verification anywhere.

Prinsip Arsitektur: Hollywood Principle

Don't call us, we'll call you — membalikkan arah kendali dependensi sistem.

PRINSIP REKAYASA PERANGKAT LUNAK

Inversion of Control: Logika bisnis fokus mengeksekusi use case; framework/assembler yang menyediakan instrumen.
Program to Interfaces: Modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah.
Evolvability: Kemampuan menukar provider atau infrastruktur tanpa memicu modifikasi pada logika inti domain.

Dependency Injection adalah perwujudan konkret dari Dependency Inversion Principle. Dengan membalik kontrol pembuatan objek, sistem menjadi adaptif terhadap perubahan zaman.

Invert the control of construction to protect domain logic from infrastructure churn.

Pisahkan perakitan dari pemakaian; isolasi logika dari infrastruktur.

Separate configuration from use: outside world assembles, inner logic executes.

Software Engineering Lab 16 – Dependency Injection

internal/di/processor.go • internal/di/gateway.go • internal/di/locator.go
cmd/demo/main.go • tests/processor_test.go • content/02-master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection`

Lukman / lukman-ss

Commit: 8678a90 | Go 1.22 Standard Library
Test Suite: 6 cases passing, 0 failures, 100% deterministic.
Race Detector: Clean pass (go test -race ./...)
Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection